

# Programación Para Sistemas.

## Examen Convocatoria Ordinaria - Enero 2026

IP: \_\_\_\_\_ Matrícula: \_\_\_\_\_ Nombre: \_\_\_\_\_ Hora: \_\_\_\_\_

- Conecta tu portátil a la WIFI Eduroam. Escribe tu matrícula, nombre (todas las letras en MAYÚSCULAS) e IP (debe empezar por 138.100, busca en: <https://www.cual-es-mi-ip.net>).
- Baja el código de apoyo de Moodle.
- La entrega es en Deliverit. Puedes entregar varias veces, la última entrega será la que se corrija. Es obligatorio que la entrega compile.
- El examen dura **90 minutos**. Escribe la hora de la última entrega en este enunciado y entrégalo.
- Durante el examen se puede utilizar apuntes y está permitido el uso de man de linux. Se considerará fraude académico la utilización de buscadores, ChatGPT o similares y compartir información con otras personas.

como  $(u, v)$ , siendo  $u$  el nodo origen y  $v$  el nodo destino.

### Se pide:

Tu trabajo consiste en escribir: `arcos.c` y `adyacencia.c`, teniendo en cuenta todos los detalles necesarios para escribir la solución que se indican en los apartados siguientes.

### Organización del código fuente

```
.
+-- data
|   +-- (archivos para pruebas)
+-- include
|   +-- arcos.h
|   +-- adyacencia.h
+-- arcos.c
+-- adyacencia.c
+-- main.c
```

## Ejercicio de C

Implementar en C distintas funciones que sirven para manipular **grafos** usando *conjuntos de arcos* (o aristas) y *matrices de adyacencia*. Para definir los grafos se utilizarán los datos en el canal de entrada estándar o en un archivo de texto. En general, antes de leer los datos no se conocerá el número de arcos (o nodos).

Un grafo dirigido,  $G = (V, E)$ , está formado por un conjunto de nodos (o vértices)  $V$  y un conjunto de arcos dirigidos  $E \subseteq V \times V$ , siendo  $V \times V$  el conjunto de todos los pares ordenados de vértices posibles (producto cartesiano). Un arco se expresa

### Compilación

El programa se debe poder compilar correctamente con los archivos colocados tal y como se muestra en el diagrama de la sección anterior y con las opciones: `-ansi -pedantic -Iinclude -Wall -Wextra -Werror`.

### Requisitos no funcionales

1. **No** es necesario **comprobar las precondiciones** (en algunos casos directamente no se pueden comprobar...).
2. **No** es necesario **comprobar** si las reservas de **memoria dinámica** se han producido correctamente.

### Requisitos funcionales

**Funciones en arcos.h** En el archivo `arcos.h` se define un `struct` para guardar los arcos del grafo, contiene dos campos: `u` y `v`, respectivamente los nodos origen y destino del arco.

`void leer_arcos(arco_t e[], int * psize);` (2 Puntos) Lee un conjunto de arcos de la entrada estándar y lo guarda en el array `e`. El número de arcos se guarda en el entero apuntado por `psize`.

1. En la entrada estándar, para cada arco, se escriben el nodo origen, un espacio y el correspondiente nodo destino, seguido por un salto de línea.
2. La lectura termina cuando alguno de los nodos es negativo, es decir, `u` ó `v` son negativos.

`arco_t* d_leer_arcos(const char * filename, int * psize);` (2 Puntos) Lee un conjunto de arcos y lo guarda en un *array* dinámico que se retorna como resultado. Los datos se leen desde un archivo cuya ruta es `filename`. El número de arcos se guarda en el entero apuntado por `psize`.

1. La lectura de los arcos debe acabar si alguno de los nodos es negativo, si se produce algún error en el formato de entrada o se alcanza el final de archivo. El formato es igual que en la función anterior.
2. El tamaño **inicial** del array dinámico se fijará en **8 arcos**, cada vez que el número de arcos supere la capacidad del array se duplicará la capacidad del mismo (con `realloc`, estrategia de *duplicación*).

## Funciones en adyacencia.h

**size\_t num\_nodos(arco\_t \* e, int size);** (1 Puntos) Calcula y retorna el número de nodos,  $N$ , en un grafo dado por el conjunto de arcos en el array **e**. Siendo **size** el número de arcos.

1. Los nodos del grafo se numeran en el intervalo  $0..N - 1$ , siendo  $N$  el número de nodos.
2. El número de nodos del grafo se puede calcular encontrando el máximo valor de todos los nodos origen y destino (en el array **e**). Es decir,  $N = \max(i \in V) + 1$ .

**unsigned char\*\* adyacencia(arco\_t \* e, int size, size\_t \* pN);** (1.5 Puntos) Genera una matriz dinámica que guarda la matriz de adyacencia de un grafo dado por el conjunto de arcos en el array **e**. Siendo **size** el número de arcos. Guarda en el entero apuntado por **pN** el número de nodos del grafo.

1. Su matriz de adyacencia de un grafo es una matriz  $A$  de tamaño  $N \times N$ , siendo  $N$  el número de nodos, donde  $A[u][v] = 1$  si existe el arco entre los nodos  $u$  y  $v$  (arco  $u \rightarrow v$ ) y 0 en caso contrario.
2. La matriz dinámica debe formarse como un array de punteros, cada uno de ellos apuntando a una fila de la matriz.
3. Se recuerda que el tipo **unsigned char** se puede usar como un entero sin signo cuyo tamaño es un byte, se usa como número no como letra.

**int es\_bidireccional(unsigned char \*\* A, size\_t N);** (1 Puntos) Retorna 1 (verdadero) si se cumple que el grafo dado por la matriz de adyacencia **A** y número de nodos **N**, es bidireccional. Retorna 0 (falso) en caso contrario. Un grafo es bidireccional si se cumple que para cualquier arco  $u \rightarrow v$  existe el arco  $v \rightarrow u$ . Es decir, la matriz de adyacencia es simétrica.

**void liberar(unsigned char \*\* A, size\_t N);** (0.5 Punto) Libera la memoria dinámica reservada para una matriz de adyacencia que se haya generado usando la función **adyacencia**. Donde **A** es la matriz dinámica y **N** el número de nodos.

## Comprueba el resultado

Comprueba el resultado compilando y ejecutando el programa principal que se proporciona. Además del programa principal se proporcionan archivos con ejemplos de entrada y la salida esperada para cada uno de ellos. El archivo **data/stdin.txt** es un ejemplo de datos en la entrada estándar.

## Ejercicio de bash

(2 Puntos) Escribe un script en **bash** llamado **gg** que genere un **grafo aleatorio** con **NODOS** (etiquetados de 0 a **NODOS**-1) y **ARCOS**. Debe aceptar la opción **-B** para indicar que el grafo es **bidireccional** (añadiendo automáticamente el arco inverso por cada arco generado). El **formato de salida** será: por cada arco, imprimir **u** (origen), un espacio, **v** (destino) y un **salto de línea**.

USO: **gg [-B] NODOS ARCOS**

### Parámetros

- **-B**: si está presente, el grafo es bidireccional (para cada **u** y **v** se imprime también **v u**).
- **NODOS**: número total de nodos (entero mayor o igual que 0).
- **ARCOS**: número de arcos a generar (entero mayor o igual que 0).

### Detalles

- Si hay algún error en los parámetros el script deberá terminar con un **mensaje** en la **salida de error** con información de su **uso** y un **exit status 1**.

- Los nodos son enteros en el rango  $[0, \text{NODOS} - 1]$ .
- Se permiten *lazos* (**u u**).
- No es necesario evitar duplicados.

### Ayuda para generar arcos aleatorios

Para generar arcos aleatorios de manera sencilla y sin dependencias externas, utiliza la variable interna de bash **\$RANDOM** y la *expansión aritmética* de Bash, por ejemplo:

```
# Suponiendo que NODOS es un entero > 0
u=$(( RANDOM % NODOS ))
v=$(( RANDOM % NODOS ))
...
echo "$u $v"
```

Para el bucle de generación de **ARCOS** puedes usar la sintaxis de Bash

```
for name in ...
```

combinado con el comando **seq** **0 \$((ARCOS - 1))** (imprime en la salida estándar líneas con números entre 0 y **ARCOS** - 1) y la sustitución de comandos (*command substitution*) de Bash **\$(command)** o **'command'** (expande la salida estándar de un comando). Consulta el manual de Bash y de **seq**.